

Actividad 201: Reconocimiento de código embebido y sintaxis

Explica brevemente cómo interactúan el servidor web y el intérprete de PHP cuando un cliente solicita una página `.php` en comparación con una página `.html` estática.

Cuando un cliente solicita una página `.php` lo que hace es una petición al servidor web, el servidor lo envía al intérprete, este lo interpreta y genera un resultado en código HTML, y el cliente lo ve en el navegador.

Cuando el cliente solicita una página en HTML, el servidor web busca el HTML y lo envía al navegador directamente.

Identifica y corrige los errores de sintaxis y de etiquetas en el siguiente fragmento de código:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi página</title>
</head>
<body>
  <h1>Bienvenido</h1>
  <?php
    echo "Hola mundo ";
    $nombre = "Carlos";
  ?>
  <p>Usuario: <?= $nombre; ?></p>
</body>
</html>
```

Actividad 202: Variables, tipos de datos y operadores

Crea una calculadora de presupuesto de compra en un archivo `presupuesto.php`:

1. Declara las siguientes variables con sus tipos correspondientes:
 - **Producto:** cadena de texto (`string`)
 - **Precio unitario:** número flotante (`float`)
 - **Cantidad:** entero (`int`)
 - **Aplica descuento:** booleano (`bool`)
2. Utiliza **operadores aritméticos y de asignación** para calcular:
 - El subtotal (`precio * cantidad`).

- El descuento del 10% si la variable booleana es `true` (usando un operador ternario u operador lógico).
 - El total final sumando un IVA del 21%.
3. Muestra una ficha de resumen en HTML utilizando sintaxis corta de PHP (`<?= ?>`) para embeber cada dato en una tabla HTML.

```
tema2 > presupuesto.php > html > body > table
1  <?php
2
3  $producto = "Galletas";
4  $precioUnitario = 5.00;
5  $cantidad = 2;
6  $aplicaDescuento = true;
7  $precioDescuento = 0;
8
9  $subtotal = $precioUnitario * $cantidad;
10
11 if($aplicaDescuento == true){
12     $precioDescuento = $subtotal * 0.1 ;
13 }
14
15 $subtotal = $precioUnitario - $precioDescuento;
16 $iva= $subtotal * 0.21;
17 $precioFinal = $subtotal + $iva;
18
19 ?>
20
21 <!DOCTYPE html>
22 <html lang="en">
23 <head>
24     <meta charset="UTF-8">
25     <meta name="viewport" content="width=device-width, initial-scale=1.0">
26     <title>Presupuesto</title>
27 </head>
28 <body>
29     <table border="1">
30         <thead>
31             <tr>
32                 <td>PRODUCTO</td>
33                 <td>PRECIO</td>
34                 <td>CANTIDAD</td>
35                 <td>DESCUENTO</td>
36                 <td>IVA</td>
37                 <td>TOTAL</td>
38             </tr>
39         </thead>
40         <tbody>
41             <tr>
42                 <td><?= $producto; ?></td>
43                 <td><?= $precioUnitario; ?></td>
44                 <td><?= $cantidad; ?></td>
45                 <td><?= $precioDescuento; ?></td>
46                 <td><?= $iva; ?></td>
47                 <td><?= $precioFinal; ?></td>
48             </tr>
49         </tbody>
50     </table>
51
52
53 </body>
54 </html>
```

Actividad 203: Ámbitos de variables (Scope)

Analiza el siguiente código, predice la salida de cada instrucción `echo` y justifica tu respuesta en función del ámbito de cada variable (local, global, superglobal o estática):

```
<?php
$localVar = "Soy local";

function probarAmbito() {
    $localVar = "Soy local";
    global $globalVar;
    static $contador = 0;

    $contador++;

    echo $localVar . "<br>";
    echo $globalVar . "<br>";
    echo "Contador: " . $contador . "<br>";
}

probarAmbito();
probarAmbito();

// ¿Qué ocurre si intentamos acceder a estas variables aquí?
// echo $localVar;
// echo $contador;
?>
```

Las variables `$localVar` y `$contador`, nos dan error, porque están dentro de la función y una vez que se sale de la función se mueren.

`$contador` al ser estático solo se inician una vez, y no se destruyen, por eso siempre imprime 0

`$localVar` se va acumulando y por eso imprime 1,2,3,4....

Actividad 204: Extracción de Nombre de Usuario desde un Email

Dado un correo electrónico completo (por ejemplo, "usuario.desarrollador@empresa.com"), escribe un script en PHP que extraiga únicamente la parte local (el nombre de usuario previo al símbolo @) y el dominio de forma independiente.

```

tema2 > actividad 204.php > ...
1  <?php
2
3  //creo la variable
4
5  $email = "usuario.desarrollador@empresa.com";
6
7  // explode separa en este caso hasta el @ la variable $email.
8
9  $partes = explode("@", $email);
10 // cada parte separada es un array, con lo cual cojo cada array para sacar la informacion.
11 $usuario = $partes[0];
12 $dominio = $partes[1];
13
14 // imprimo
15 echo "usuario: " . $usuario. "<br>";
16 echo "dominio: " . $dominio. "<br>";
17
18
19 ?>

```

Actividad 205: Normalización y Limpieza de Cadenas

Un usuario introduce un nombre con espacios sobrantes al inicio y final, y con combinación desordenada de mayúsculas y minúsculas (ejemplo: " jUaN cArLoS pErEz "). Crea una función en PHP que limpie los espacios y devuelva el nombre con la primera letra de cada palabra en mayúscula y el resto en minúscula ("Juan Carlos Perez").

```

tema2 > actividad 205.php > ...
1  <?php
2
3  $nombre = " jUaN cArLoS pErEz ";
4
5  function limpieza ($nombreSucio){
6      // se quitan los espacios
7      $sinEspacios = trim($nombreSucio);
8      // quitas los espacios
9      $sinEspaciosMin = strtolower($sinEspacios);
10     //La primera letra en mayusculas
11     $nombreLimpio = ucwords($sinEspaciosMin);
12     // imprimo
13     echo $nombreLimpio;
14 }
15
16 // llamo a la función
17
18 limpieza($nombre);
19
20
21 ?>

```

Actividad 206: Enmascaramiento de Tarjeta de Crédito (Slicing Negativo)

Crea una función llamada `enmascararTarjeta($numeroTarjeta)` que reciba un número de tarjeta de 16 dígitos (como cadena o entero) y devuelva una cadena donde todos los dígitos excepto los últimos 4 estén reemplazados por asteriscos * (ejemplo: "*****1234").

```
actividad 206.php > ...
1  <?php
2
3  function enmascararTarjeta($numeroTarjeta){
4
5  //primero limpiar el numero, no espacios, no simbolos, solo numeros
6
7  $numeroLimpio = preg_replace('/\D/', '', $numeroTarjeta);
8
9  // ponemos los asteriscos menos en los cuatro ultimos digitos
10
11 $asteriscos = str_repeat('*',strlen($numeroLimpio) - 4);
12
13 // para que devuelva los 4 ultimos digitos de la cadena
14
15 return $asteriscos . substr($numeroLimpio, -4);
16
17
18
19 }
20
21 echo enmascararTarjeta('1234 5675 2589 2458');
22 echo "\n";
23 echo enmascararTarjeta('1111-2222-3333-4444');
24 echo "\n";
25 echo enmascararTarjeta('1111222233334444');
26
27 ?>
```

Actividad 207: Censo de Palabras y Censura de Contenido

Dada una frase introducida en un blog o foro y una lista de palabras prohibidas, reemplaza cualquier palabra prohibida por el texto "[CENSURADO]" y muestra cuántas palabras totales contiene la frase final.

```
actividad 207.php > ...
1  <?php
2
3  function censurarContar ($frase, $censura){
4
5  // se reemplazan las palabras prohibidas
6  $fraseCensurada = str_ireplace($censura, '[CENSURADO]', $frase);
7
8  // se cuentan las palabras que tiene la frase
9  $palabras = explode(' ', $fraseCensurada); // guarda en un array cada palabra que este separada por un espacio
10 $total = count($palabras); // cuenta las palabras
11
12 // se muestra el resultado
13 echo $fraseCensurada . "\n";
14 echo "Hay un total de: " . $total . " de palabras.";
15
16 }
17
18 $frase = 'Los foros de mierda, estan llenos de subnormales que van de listos.';
19 $prohibidas = ['mierda','subnormales'];
20
21 censurarContar($frase, $prohibidas);
22
23
24 ?>
```

Actividad 208: Conversión de Formato de Cadena a Array y Formateo CSV

Dado un string con una lista de tecnologías separadas por comas (ejemplo: "php, javascript, html, css, sql"), conviértelo en un array, limpia los espacios de cada elemento, convierte cada palabra a mayúsculas y finalmente genera una cadena con formato separado por guiones ("PHP - JAVASCRIPT - HTML - CSS - SQL").

Paso a paso:

1. `explode(",", $listaString)` corta la cadena en trozos cada vez que encuentra una coma y retorna un array.
2. Mediante `array_map()`, aplicamos a cada elemento individual `trim()` (para quitar el espacio después de la coma) y `strtoupper()` (para convertirlo a mayúsculas).
3. `implode(" - ", $elementosProcesados)` toma el array procesado y lo une de nuevo en un solo string utilizando " - " como delimitador.

actividad 208.php > PHP Intelephense > conversion

```
1  <?php
2
3  function conversion ($listaString){
4
5      /*1.explode(",", $listaString) corta la cadena en trozos cada vez que encuentra una coma
6      y retorna un array.*/
7
8      $arrayLista = explode(',',$listaString);
9
10
11     /*2.Mediante array_map(), aplicamos a cada elemento individual trim()
12     (para quitar el espacio después de la coma) y strtoupper() (para convertirlo a mayúsculas).*/
13
14     $sinEspacios = array_map('trim', $arrayLista);
15     $elementosProcesados = array_map('strtoupper', $sinEspacios);
16     /* como trim() y strtoupper() solo funcionan con String, hay que ponerlo así, si no da error */
17
18     /*3.implode(" - ", $elementosProcesados) toma el array procesado y lo une de nuevo en un
19     solo string utilizando " - " como delimitador*/
20
21     $arrayDefinitivo = implode(" - ", $elementosProcesados);
22
23     /*5.Imprimimos*/
24
25     return $arrayDefinitivo;
26
27
28 }
29
30 echo conversion("php, javascript, html, css, sql");
31
32 >>
```